

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUA – PEC 1

Presentación

Esta primera PEC sirve como introducción a las arquitecturas de software para el desarrollo de aplicaciones distribuidas donde se trabajan los cimientos, técnicas y habilidades aplicables en la definición moderna de arquitecturas del software y se profundiza en la selección del estilo arquitectónico más adecuado según la tipología de la aplicación a desarrollar.

Para elaborar la actividad se formulan cuatro ejercicios: En el primer ejercicio se evaluará alguno de los aspectos fundamentales en la definición de una arquitectura, mientras que el segundo y el tercer ejercicio habrá que decidir y razonar el estilo arquitectónico (o mezcla de estilos) más adecuado en función de los requisitos planteados en un caso de estudio. Finalmente, el último ejercicio evaluará algunos conceptos básicos de las arquitecturas distribuidas.

La PEC 1 alcanza los contenidos del libro *Fundamentals of Software Architecture* (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta PEC 1 se trabaja la siguiente competencia del Grado en Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.

Objetivos

Los objetivos de la PEC 1 son:

- Explicar los fundamentos de la arquitectura de software y las características de una arquitectura.
- Exponer argumentos sobre los puntos fuertes y los puntos débiles de los diferentes estilos arquitectónicos y cómo estos influyen en la decisión del estilo o estilos arquitectónicos más adecuados para una determinada aplicación.

- Situar los nuevos paradigmas emergentes de la ingeniería del software dentro del marco de las arquitecturas del software.

Descripción de la PEC a realizar

Ejercicio 1 (2 puntos)

Realizad un estudio comparativo de los estilos arquitectónicos *Layered Architecture* y *Microservices Architecture* en función de sus características. Usad la tabla mostrada a continuación.

Por último, describid una **decisión arquitectónica** específica para cada uno de los dos estilos anteriores que suponga una regla o restricción técnica que el equipo de desarrollo debería seguir para mantener la integridad del estilo elegido.

	<i>Layered Architecture</i>	<i>Microservices Architecture</i>
<i>Partitioning type</i>		
<i>Number of quanta</i>		
<i>Deployability</i>		
<i>Elasticity</i>		
<i>Evolutionary</i>		
<i>Fault tolerance</i>		
<i>Modularity</i>		
<i>Overall cost</i>		
<i>Performance</i>		
<i>Reliability</i>		
<i>Scalability</i>		
<i>Simplicity</i>		
<i>Testability</i>		

Solución

	<i>Layered Architecture</i>	<i>Microservices Architecture</i>
<i>Partitioning type</i>	Técnico. Se separa por roles tecnológicos como presentación, negocio y persistencia	Dominio. Se organiza en torno a contextos acotados (<i>bounded contexts</i>) de negocio
<i>Number of quanta</i>	1. Todo el sistema es una única unidad de despliegue con una base de datos	Múltiples. Cada servicio es una unidad independiente con su propia base de datos y características

<i>Deployability</i>	Baja. Un cambio mínimo obliga a volver a desplegar todo el monolito	Muy alta. Unidades pequeñas y desacopladas permiten despliegues independientes y frecuentes
<i>Elasticity</i>	Muy baja. La naturaleza monolítica dificulta el autoescalado ante picos de demanda	Muy alta. La integración con el Cloud permite escalar servicios específicos según la carga
<i>Evolutionary</i>	Muy baja. El alto acoplamiento entre capas dificulta la evolución tecnológica rápida	Muy alta. El desacoplamiento físico y lógico favorece el cambio incremental por servicio.
<i>Fault tolerance</i>	Baja. Un error en una parte puede hacer caer toda la aplicación	Alta. El aislamiento físico evita que la caída de un servicio afecte a todo el sistema
<i>Modularity</i>	Baja. No se considera modular debido a su alto acoplamiento interno	Máxima. La modularidad es su fundamento mediante servicios separados por dominio
<i>Overall cost</i>	Bajo. Es sencillo de construir y mantener para aplicaciones pequeñas	Muy elevado. Requiere infraestructura compleja y una cultura DevOps madura
<i>Performance</i>	Media/Baja. Puede sufrir por la dependencia excesiva de llamadas entre capas ("sinkholes").	Baja (Variable). Se ve afectado por la latencia de red y la seguridad en cada endpoint
<i>Reliability</i>	Media. Menos uso de la red que los sistemas distribuidos, pero con riesgos sistémicos.	Alta. Se consigue mediante técnicas de redundancia y <i>fail-over</i> .
<i>Scalability</i>	Muy baja. Escalar un monolito es caro y técnicamente muy complejo	Muy alta. Permite gestionar millones de usuarios concurrentes de forma eficiente
<i>Simplicity</i>	Alta. Es fácil de entender e implementar por equipos pequeños y poco experimentados.	Muy baja. Es inherentemente compleja por el número de componentes y
<i>Testability</i>	Baja. Los cambios suelen requerir una regresión completa del monolito.	Alta. Permite realizar pruebas automáticas aisladas para cada servicio.

Un ejemplo de decisión arquitectónica específica para **Layered Architecture** podría ser la siguiente: "Se restringe a la capa de **presentación** el acceso directo a la capa de **persistencia**; cualquier petición de datos debe pasar obligatoriamente por la capa de **negocio**". Esta restricción garantiza el concepto de **aislamiento entre capas**, permitiendo que los cambios en la base de datos no impacten directamente en la interfaz de usuario.

Un ejemplo de decisión arquitectónica específica para **Microservices Architecture** podría ser la siguiente: *"Cada microservicio debe poseer su propio **esquema de base de datos aislado e independiente**; queda prohibido que un servicio acceda directamente a las tablas de otro servicio"*. Esta restricción asegura el cumplimiento del contexto acotado y permite que cada servicio evolucione y escale de forma independiente sin acoplamiento de datos.

Ejercicio 2 (3 puntos)

Photo&Film4You, una tienda de material audiovisual especializada en equipos para rodajes de películas y sesiones profesionales de fotografía, pretende añadir una nueva funcionalidad para recuperar contenido audiovisual generado con tecnologías del siglo XX, implementando un flujo de procesamiento estructurado por etapas consecutivas. El cliente deja físicamente el medio antiguo, en formatos como VHS, Betamax, DVD, CD o cintas de audio, en una oficina de Photo&Film4You. En un primer paso, se añade una pegatina con un código QR que identifica de manera única cada medio, y los discos y cintas se almacenan en armarios diseñados para protegerlos.

A continuación, los operarios retiran los medios de los armarios e introducen cada medio (soporte) en el único equipo de digitalización disponible, donde se procesa de manera ordenada y se genera una salida en un formato estándar de video o audio, que se almacena directamente en un sistema de almacenamiento de objetos (Amazon S3). Cada medio sigue un camino definido de principio a fin, y el equipo indica mediante una señal luminosa cuando ha finalizado el procesamiento de un medio. En ese momento, el medio se extrae, se verifica mediante el código QR y se guarda en el armario de medios digitalizados, mientras que el siguiente medio inicia su propio procesamiento siguiendo exactamente el mismo recorrido.

En las fases posteriores, un operador revisa y ajusta manualmente el medio digitalizado según sea necesario. Finalmente, una vez completadas todas las transformaciones y revisiones, el video o audio queda disponible en su formato digital final, listo para almacenamiento o uso posterior, y el usuario puede acceder al resultado directamente en la tienda online de Photo&Film4You, dentro de su área de pedidos.

¿Qué estilo arquitectónico del “Capítulo II: Estilos Arquitectónicos” del libro *Fundamentals of Software Architecture* crees que se adapta mejor a los nuevos requisitos relacionados con la digitalización de medios antiguos, como VHS, Betamax, DVD, CD y cintas de audio?

Justifica tu respuesta comparando los nuevos requisitos y restricciones del proyecto con las ventajas y desventajas del estilo arquitectónico que elijas.

NOTA: En este ejercicio no se debe analizar el estilo arquitectónico del sistema encargado de gestionar el catálogo de material para alquiler, sino únicamente el de este nuevo subsistema.

Solución

El estilo arquitectónico que mejor se ajusta a esta funcionalidad es el Pipeline Architecture Style. Este enfoque es especialmente adecuado porque el proceso de recuperación y digitalización de contenidos físicos se desarrolla como una secuencia de etapas claramente definidas, en las que la salida de una etapa sirve como entrada para la siguiente. Desde la recepción física del medio, su etiquetado con código QR y almacenamiento seguro, hasta la digitalización en el equipo disponible, cada paso puede considerarse un componente independiente del flujo, transformando el medio de una etapa a otra y generando un procesamiento continuo y ordenado. La naturaleza secuencial y estructurada del pipeline facilita la gestión de recursos limitados, como el único equipo de digitalización, y permite mantener un control claro sobre qué medio se está procesando en cada momento.

Además, este flujo permite integrar la revisión y validación final por parte de un operador humano, quien garantiza la calidad del contenido digitalizado antes de almacenarlo. La salida final en el sistema de almacenamiento de objetos (S3) y la disponibilidad del contenido en la tienda online se integran como la etapa final del pipeline, completando un flujo coherente, supervisable y eficiente.

Aunque este enfoque es ideal para procesos lineales y predecibles, presenta algunas limitaciones. Su naturaleza secuencial implica que cualquier retraso o problema en una etapa puede bloquear el flujo completo. Manejar excepciones o medios problemáticos requiere coordinación cuidadosa, y el pipeline ofrece poca flexibilidad para reordenar etapas o incorporar nuevas transformaciones sin afectar todo el flujo. En resumen, es un modelo adecuado para garantizar un proceso ordenado y controlado, pero requiere una supervisión estricta y una gestión eficiente de los recursos disponibles.

Ejercicio 3 (3 puntos)

Se propone, sobre la misma funcionalidad de recuperación de contenidos generados en medios físicos, analizada en el ejercicio 2, una modificación de los flujos de funcionamiento con el objetivo de mejorar la productividad. Gracias a una inversión extraordinaria, la oficina de Photo&FilmYou ahora cuenta con 10 equipos de digitalización, lo que permite procesar los medios de forma más eficiente. El cliente dejará físicamente el medio antiguo en los mismos formatos contemplados en el ejercicio anterior, con el fin de digitalizarlo y mejorar su calidad.

Cuando un cliente deja su medio físico, se añade una pegatina con un código QR que lo identifica de forma única. Tras la recepción, los discos y cintas se almacenan en armarios diseñados para protegerlos. El procesado y almacenamiento de los medios digitalizados se mantiene respecto al ejercicio anterior. Aunque en ciertos momentos existe intervención humana, que puede implicar que los flujos queden momentáneamente parados, la incorporación de las 10 máquinas permite paralelizar tareas, de modo que varios medios se procesan simultáneamente. Cada equipo indica

mediante una señal luminosa cuando finaliza el procesamiento de un medio; en ese momento, el medio se extrae, se lee el código QR para identificarlo y se guarda en el armario de medios digitalizados, mientras que el siguiente medio se introduce en el equipo disponible, manteniendo un flujo constante y eficiente de digitalización.

Posteriormente, los agentes de IA analizan cada parte del recurso digitalizado para detectar posibles deficiencias de calidad y aplicar mejoras automáticas cuando sea necesario. No obstante, en algunas ocasiones la IA puede generar “alucinaciones”, realizando modificaciones proactivas que no siempre son correctas. Por ello, un operador de calidad participa en las fases finales del proceso, revisando y ajustando manualmente los fotogramas que la IA no haya podido mejorar adecuadamente o que presenten inconsistencias, utilizando en caso necesario la digitalización en crudo generada en las fases iniciales como referencia para garantizar la fidelidad del contenido. Una herramienta de gestión proporciona de manera visual, sobre el perfil de este operador, un seguimiento de las notificaciones de los distintos medios que va recibiendo, facilitando la priorización y el control del trabajo pendiente. Finalmente, una vez completadas las mejoras automáticas validadas y la intervención del operador de calidad, el vídeo o audio queda disponible en su formato digital final, listo para almacenamiento o uso posterior, y el usuario puede ver el resultado directamente en la tienda online de Photo & Film4You dentro de sus pedidos, accediendo a la versión digital de su contenido.

¿Qué estilo arquitectónico del “Capítulo II: Estilos Arquitectónicos” del libro *Fundamentals of Software Architecture* crees que se adapta mejor a los nuevos requisitos relacionados con la digitalización de medios antiguos, como VHS, Betamax, DVD, CD y cintas de audio?

Justifica tu respuesta comparando los nuevos requisitos y restricciones del proyecto con las ventajas y desventajas del estilo arquitectónico que elijas.

NOTA: En este ejercicio no se debe analizar el estilo arquitectónico del sistema encargado de gestionar el catálogo de material para alquiler, sino únicamente el de este nuevo subsistema.

Solución

Tras una primera aproximación basada en un flujo estrictamente secuencial, analizada en el segundo ejercicio y alineada con un estilo arquitectónico de tipo *pipeline*, la evolución del proyecto y la incorporación de nuevos requisitos hacen que resulte más adecuado adoptar un enfoque basado en Event-Driven Architecture (EDA). La reciente inversión en nuevos equipos de digitalización incrementa la capacidad operativa, pero también introduce mayor complejidad en la coordinación: ahora existen múltiples puntos donde pueden finalizar procesos en momentos distintos, generando estados intermedios que deben gestionarse de forma independiente. La finalización de cada digitalización, la verificación del código QR, la validación del contenido por parte del operador y su posterior publicación son sucesos que ocurren de manera desacoplada y no necesariamente sincronizada, lo que refuerza la necesidad de un modelo orientado a eventos.

Además, parte del flujo presenta un comportamiento claramente reactivo: el sistema debe responder cuando un equipo libera capacidad, cuando un medio cambia de estado o cuando el operador valida el contenido digital. En un enfoque pipeline tradicional, esta coordinación paralela exigiría mecanismos adicionales de control y sincronización, aumentando el acoplamiento y la complejidad interna. En cambio, la EDA permite que cada componente emita y consuma eventos relacionados con cambios de estado, facilitando la gestión de múltiples equipos trabajando en paralelo, mejorando la resiliencia ante fallos puntuales y permitiendo incorporar nuevos procesos o consumidores de eventos sin rediseñar todo el flujo. En este contexto, el modelo orientado a eventos se ajusta mejor a un entorno más dinámico, concurrente y operativo que el inicialmente planteado.

Entre las ventajas de aplicar EDA en este escenario destacan la flexibilidad y la capacidad de gestionar múltiples flujos concurrentes derivados de la incorporación de varios equipos de digitalización trabajando en paralelo. Cada finalización de proceso, liberación de equipo o validación por parte del operador se modela como un evento independiente, lo que permite que el sistema reaccione de forma desacoplada y coordinada sin imponer un orden estrictamente lineal. Asimismo, este enfoque facilita la extensibilidad futura. Por ejemplo, para incorporar nuevos tipos de medios, nuevos puntos de control o servicios adicionales sin necesidad de rediseñar por completo la arquitectura existente.

Como contrapartida, la arquitectura orientada a eventos introduce una mayor complejidad en la gestión y orquestación de los eventos, especialmente en lo relativo a la trazabilidad y consistencia del estado de cada medio a lo largo del proceso. Es necesario definir claramente los eventos, sus contratos y los mecanismos de seguimiento para evitar pérdidas de información o estados inconsistentes, más aún cuando intervienen tanto procesos automáticos como validaciones humanas. Sin embargo, pese a este incremento en la complejidad técnica, los beneficios en términos de escalabilidad, resiliencia y capacidad de reacción ante cambios de estado hacen que este enfoque sea especialmente coherente con la evolución operativa y la mayor concurrencia del sistema.

Ejercicio 4 (2 puntos)

Responded brevemente a las siguientes preguntas sobre los conceptos vistos en el libro *"Fundamentals of software architecture"*.

4.1 (0.5 puntos) La frontera entre las decisiones arquitectónicas y las decisiones de diseño muchas veces son complicadas y la frontera entre una y otra no está definida de forma nítida. Catalogad la decisión *"Elegir entre una base de datos NoSQL (MongoDB) o una relacional (PostgreSQL) para el sistema de alquiler"* entre decisión arquitectónica o decisión de diseño y argumentalo usando los tres criterios estudiados que os pueden ayudar a determinar si una decisión se sitúa más en el lado de la arquitectura o del diseño.

Solución

La decisión se considera **arquitectónica** por los siguientes motivos:

- **Naturaleza de la decisión:** Es estratégica y a largo plazo; define cómo se gestionará la consistencia y estructura de los datos del sistema
- **Nivel de esfuerzo:** Se considera arquitectura "*aquello que es difícil de cambiar*"; migrar toda la persistencia una vez el sistema está en producción de un base de datos relacional a NoSQL o a la inversa requiere un esfuerzo muy elevado.
- **Importancia de los 'trade-offs' de la elección:** La elección implica compromisos críticos (por ejemplo en términos de escalabilidad y consistencia) que impactan la estructura global

Véase el Capítulo 2. "*Architectural Thinking - Architecture Versus Design*" del libro "*Fundamentals of Software Architecture*".

4.2 (0.5 puntos) ¿Cuáles de estas tareas son responsabilidades de un arquitecto y cuáles son responsabilidad del equipo de desarrollo?

- Negociar con los stakeholders para conseguir la aprobación de una decisión que aumenta los costes operativos a corto plazo, pero mejora la escalabilidad.
- Optimizar la lógica interna y el uso de memoria de un algoritmo de búsqueda dentro de un componente de negocio específico.
- Evaluar la viabilidad de la arquitectura actual frente al impacto de nuevas tendencias tecnológicas, como la integración de servicios de IA generativa.
- Codificar un recubrimiento para una librería de terceros con el fin de simplificar su uso por parte de otros programadores del equipo.
- Definir las funciones automáticas para monitorizar que el acoplamiento entre servicios no supere los límites establecidos.
- Configurar el formato de las trazas de depuración para un módulo concreto del sistema

Solución

- *Negociar con los stakeholders para conseguir la aprobación de una decisión que aumenta los costes operativos a corto plazo, pero mejora la escalabilidad.* **Arquitecto:** Un arquitecto debe poseer habilidades interpersonales y de negociación para entender la política de la empresa y justificar decisiones técnicas que a menudo son rechazadas por otros departamentos.
- *Optimizar la lógica interna y el uso de memoria de un algoritmo de búsqueda dentro de un componente de negocio específico.* **Equipo de desarrollo:** La implementación detallada del código, su eficiencia interna y el diseño de las funciones son parte del "arte de la programación" y recaen en los desarrolladores.
- *Evaluar la viabilidad de la arquitectura actual frente al impacto de nuevas tendencias tecnológicas, como la integración de servicios de IA generativa.* **Arquitecto:** Es una expectativa central del arquitecto analizar continuamente el entorno tecnológico y la arquitectura definida hace años para recomendar mejoras y asegurar que siga siendo válida hoy.

- *Codificar un recubrimiento para una librería de terceros con el fin de simplificar su uso por parte de otros programadores del equipo. **Equipo de desarrollo:*** Aunque el arquitecto guía la elección de herramientas, la creación de componentes de soporte y la estructura del código diario son responsabilidad de los desarrolladores.
- *Definir las funciones automáticas para monitorizar que el acoplamiento entre servicios no supere los límites establecidos. **Arquitecto:*** El arquitecto es responsable de definir decisiones y principios, así como de asegurar el cumplimiento de estos mediante el uso de herramientas de gobernanza automatizada como las funciones automáticas de monitorización del acoplamiento.
- *Configurar el formato de las trazas de depuración para un módulo concreto del sistema. **Equipo de desarrollo:*** El arquitecto debe guiar las elecciones tecnológicas (ej. indicar que se use un framework de logging), pero son los desarrolladores quienes deben configurar la herramienta concreta para su trabajo diario.

Véase el Capítulo 2 del libro “*Fundamentals of Software Architecture*”.

4.3 (0.5 puntos) Supongamos que diseñamos un sistema de pujas en tiempo real (como una plataforma de subastas masivas online) que sigue una arquitectura *Event Driven*. Poned ejemplos concretos donde se vea que no es posible maximizar simultáneamente todas las características deseables para el sistema.

Solución

En un sistema de pujas basado en eventos como el que se describe en el enunciado se busca maximizar el **rendimiento** y la **escalabilidad** para procesar miles de ofertas concurrentes de forma asíncrona.

Esto entra en conflicto directo con la **consistencia de datos**, la **testeabilidad** y la **simplicidad**:

- **Consistencia eventual vs. inmediata:** Para mantener un rendimiento máximo, el sistema utiliza comunicación asíncrona y esto implica aceptar una consistencia eventual: un post-procesador de auditoría podría detectar que una puja llegó milisegundos después del cierre, invalidándola a posteriori. Si intentáramos garantizar consistencia inmediata (bloqueando el sistema hasta estar seguros de quién es el ganador), el *rendimiento* caería drásticamente.
- **Simplicidad y testeabilidad:** Event Driven es un estilo inherentemente complejo debido a sus flujos no deterministas. En una subasta masiva, es extremadamente difícil probar y predecir todos los escenarios posibles (pujas que llegan desordenadas, fallos de red, etc. Maximizar el rendimiento y la escalabilidad del sistema obliga a sacrificar la facilidad para realizar pruebas de regresión deterministas, ya que el orden y resultado de los eventos pueden variar dinámicamente.

Véase los Capítulos 1, 4 y 14. del libro “*Fundamentals of Software Architecture*”.

4.4 (0.5 puntos) Elegid dos expectativas que creáis que están asociadas al rol de arquitecto que se hayan debatido en el debate de la actividad, y poned un ejemplo de tarea concreta por cada

una de ellas. Los ejemplos elegidos no pueden aparecer en el libro *Fundamentals of Software Architecture*.

Solución

- **Analizar continuamente la arquitectura:** *Evaluar las alternativas de despliegue al Cloud para ver si hay que hacer una migración en este sentido.*
- **Tener cierto dominio del negocio:** *En el negocio bancario, conocer qué es una hipoteca inversa.*

Véase el Capítulo 1: “Introduction” del libro “*Fundamentals of Software Architecture*”.

Recursos

Recursos Básicos

- Libro "Fundamentals of Software Architecture" (Richards & Ford): Chapter 1. Introduction.
- Libro "Fundamentals of Software Architecture" (Richards & Ford): I Foundations (Chapters 2, 4, 5 y 8)
- Libro "Fundamentals of Software Architecture" (Richards & Ford): II Architecture Styles (Chapters 9 - 14, 16 - 18)

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta misma prueba de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que enviar un único documento PDF con las respuestas a todos los ejercicios. El nombre del archivo tiene que ser: *ApellidosNombre_ISCSD_PEC1.pdf*.

Este documento se enviará al espacio de “Entrega de la PEC 1” del aula antes de las **23:59 horas del día 16 de marzo de 2026**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación

	Sobresaliente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1 (2 p.)	Comparación correcta para 9 o más características y explicaciones correctas, decisiones correctas (2)	Comparación correcta para entre 7 y 8 características sin explicaciones o con explicaciones incorrectas, una de las dos decisiones correctas (1,5)	Comparación correcta para entre 5 y 7 características y explicaciones correctas, una de las dos decisiones correctas (1)	Comparación correcta para menos de 5 características y explicaciones correctas (0,5)	0
Ej. 2 (3 p.)	Elección arquitectónica correcta con razonamiento correcto y adecuado en función de los argumentos y restricciones del enunciado (3)	Elección arquitectónica correcta, pero razonamiento incorrecto o no adecuado en función de los argumentos y restricciones del enunciado (2)	Elección arquitectónica incorrecta, pero razonamiento adecuado en función de los argumentos y restricciones del enunciado (1,5)	Elección arquitectónica correcta, pero no razonada, (0,75)	0
Ej. 3 (3 p.)	Elección arquitectónica correcta con razonamiento correcto y adecuado en función de los argumentos y restricciones del enunciado (3)	Elección arquitectónica correcta, pero razonamiento incorrecto o no adecuado en función de los argumentos y restricciones del enunciado (2)	Elección arquitectónica incorrecta, pero razonamiento adecuado en función de los argumentos y restricciones del enunciado (1,5)	Elección arquitectónica correcta, pero no razonada (0,75)	0

Ej. 4 (2 p.)	4 respuestas correctas y bien razonadas (2)	3 respuestas correctas y bien razonadas, y 1 respuesta incorrecta o no razonada (1,5)	2 respuestas correctas y bien razonadas, y/o 2 respuestas incorrectas o no razonadas (1)	1 respuesta correcta y bien razonada, y 3 respuestas incorrectas y/o no razonadas (0,5)	0
-------------------------	---	---	--	---	---